Eg1

1.

前向模式 forward-mode：处理少输入多输出的情况

反向模式：多输入少输出的情况

2.

JAX：Google 于 2018 年开发的高性能机器学习工具

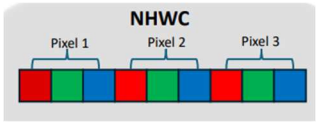
采用**函数式**编程风格，强调纯函数和不可变数组。课件将这与 PyTorch 的命令式编程、可变张量作了对比。

vmap 和 batching 有关，grad 和自动微分有关，jit 和即时编译有关。

3.

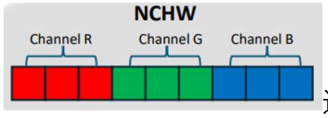
张量各个维度的排列顺序:

NHWC：Batch、Height、Width、Channels。每个像素的通道值放在一起，例如一个 RGB 像素的 R、G、B 值相邻。



改善合并内存访问

NCHW：Batch、Channels、Height、Width。同一个通道的像素数据放在一起，例如先排红色通道的像素，再排绿色通道。



适合按通道遍历图像数据

4.

反向模式自动微分中的梯度检查点，反向模式要先做前向计算，再在反向传播时求梯度。通常需要保存中间激活值；网络越深，需要保存的中间结果越多，内存占用也越高。**梯度检查点**不保存所有中间结果，而是只保留部分检查点；反向传播需要其他激活值时，再从检查点重新计算。这就用额外计算换更低的峰值内存。

**减少峰值内存，但会增加计算量**，因为反向传播时要重新计算未保存的激活值

内存使用量可降至约$\sqrt{n}$级别，计算量增加约 33%

保留反向传播；只保存部分检查点，并在反向传播时重新计算其他激活值

梯度检查点以重算换取内存

5.

Ring all-reduce 分成两个阶段：

i. **Reduce-scatter：**每个设备把数据分成 N 份，在环上逐步传递并累加。每台设备发送 N-1 次，每次发送一份，大小为$\frac{P}{N}$。因此每台设备发送：$(N-1)\frac{P}{N}$

其中 P 是要做 all-reduce 的整份数据大小， N 是设备数。

ii. **All-gather：**每台设备再把自己得到的归约结果块传给其他设备，让所有设备最终拿到完整结果。也是发送 N-1 次、每次$\frac{P}{N}$：$(N-1)\frac{P}{N}$

两个阶段相加就是$2P\frac{N-1}{N}$

6.

**Amdahl's Law (upper bound on speedup)**

- Speedup ≤ 1 / (s + (1 − s)/N), where s = serial fraction
- Example: s = 0.10 ⇒ max speedup = 10× as N → ∞
- Even small s severely caps efficiency at large N

7.

1F1B 指 **one forward, one backward**：流水线运行中交错执行前向和反向计算。

**GPipe** 是先完成所有微批次的前向计算，再做反向计算；课件说这种方式简单一致，但激活值占用较高。

8.

Which property defines dual numbers used for forward-mode AD?

$\varepsilon^2 = 0$ (nilpotency)

9.

Dual-number multiplication：(a,b)×(c,d)=(ac, ad+bc)

10.

**MCU** 是 *Microcontroller Unit*，即微控制器，也常叫单片机。

**反向模式为什么占内存：**前向计算产生中间激活值，反向传播通常需要这些值来计算梯度。网络越深，需要保留的中间结果越多；反向传播开始时，内存压力可能达到峰值。

**检查点怎么省内存：**不保存所有激活值，只保存部分检查点；反向传播时，再从检查点重新计算其他激活值。

这类设备 RAM 很有限，课件指出，激进检查点甚至完全重算可以让训练有机会装进设备内存，代价是计算量和运行时间增加。

11.

Ring all-reduce：用于数据并行训练中的梯度同步。每台设备沿环与相邻设备通信，分成 reduce-scatter 和 all-gather 两个阶段。课件称它适合**大消息**，并给出了每设备通信量的计算。

Tree all-reduce：课件提到它通常适合**小消息**：相比 ring，延迟较低。

Parameter Server pull/push：较灵活，支持异步训练，能容忍较慢的客户端；代价是可能让服务器成为瓶颈，也可能产生陈旧梯度，需要谨慎调整学习率。

12.

TensorFlow 1.x：先定义符号计算图，再通过 Session 执行（define-then-run）先完整构建计算结构，课件提到这种方式有利于全局优化和内存规划。

PyTorch：动态计算图，代码运行时构建计算过程（define-by-run）控制流程灵活，开

发和调试方便。

TensorFlow 2.x：默认 eager execution，即时执行；也可用 tf.function 平时可以即时执行，需要时再把函数转成图执行。

JAX：对纯函数使用 jit、grad、vmap 等变换采用函数式风格和不可变数组；课件提到用 XLA 做即时编译和硬件优化。

13.

TPU 用脉动阵列进行矩阵计算；把矩阵维度补齐或对齐到适合的大小（如 128 的倍数），通常能提高阵列利用率。这个具体优化属于 **Week 6**，前五周只介绍了 TPU 和脉动阵列的基本概念。

14.

张量从 NHWC 排列转换成 NCHW：

新顺序依次取原来的第 **0、3、1、2** 个维度，也就是 **[0, 3, 1, 2]**

15.

PyTorch define-by-run and dynamic computation graphs

16.

安全聚合 Secure Aggregation 使服务器只能看到客户端更新的总和；DP differential privacy 通过裁剪和添加校准噪声提供隐私保护

安全聚合隐藏单个客户端的更新；DP 添加经过校准的噪声，为隐私保护提供形式化保证。

即使使用 8-bit 量化，如果要让服务器看不到单个客户端更新，仍可使用安全聚合

联邦学习中，原始数据留在客户端；客户端本地处理数据并发送模型更新，DP 的裁剪和加噪也可以在设备上进行。

安全聚合（Secure Aggregation）：**一种保护客户端更新的聚合协议，使服务器只能得到多个客户端更新的总和或聚合结果**，而看不到每个客户端单独发来的更新。用成对掩码让掩码在总和里抵消，或使用同态加密直接聚合密文；还要考虑客户端掉线等情况。

差分隐私（Differential Privacy，DP）：加入或移除一个记录时，算法输出的变化受到限制。直观来说，单条数据不应明显改变最终发布的结果。先**裁剪**客户端更新，以限制敏感度；再添加按隐私目标校准的**高斯噪声**，并跟踪隐私预算。课件指出，裁剪和加噪可以在设备上进行，并与安全聚合结合使用。

17.

**弹性分布式训练（elastic distributed training）**：训练过程中，参与训练的 worker 可以变化，比如某些机器退出、故障，或新的机器加入。训练系统需要适应变化，重新安排数据分片和通信组，并继续训练。课件把它描述为**运行时可调整 world size 的同步数据并行**。

**World size**：当前这次分布式训练中参与工作的进程数，也就是各个 rank 的总数。实践中常对应参与训练的 GPU worker 数量。比如 8 个进程一起训练，world size 就是 8。

world size 变了之后，重新计算实际的全局 batch size，并相应调整学习率。

**Process group** 是参与集体通信的一组分布式进程。world size 或成员变化后，原来的通信组可能不再对应当前 worker 集合。成员变化时，重新 rendezvous 并按当前 worker 集合重建 process groups。

**Checkpoint（检查点）**保存训练状态，供故障恢复或弹性调整后继续训练。课件要求保存模型、优化器、随机数状态以及分片等元数据，并支持 world size 改变后的映射。when world size changes 应该保留 checkpoint，并保存必要的训练状态和分片信息，供恢复及重新分配使用。

**Loss scaling（损失缩放）**与数值精度有关，混合精度训练时需要正确处理梯度和损失缩放。不能不管精度变化就一律固定。所以 when world size changes 应该根据训练精度正确处理并保持梯度缩放、损失缩放的一致性。

worker 数量变了（world size 改变），要重建通信配置、重新分片，并重新核对全局 batch 和学习率；checkpoint 不应关闭。

18.

**FSDP** 是 **Fully Sharded Data Parallel（全分片数据并行）**。它把模型参数、优化器状态等分片存放到不同 GPU 上，以减少每张 GPU 的内存占用。计算某个模块前，各 GPU 先通过 **all-gather** 暂时收集所需参数；计算完成后，再通过 **reduce-scatter** 汇总并分发梯度。代价是需要更多通信。

参数分片保存在各个 rank 上；计算某个模块前，再 all-gather 该模块所需的参数。

FSDP 在前向计算前对所需参数分片执行 all-gather。

FSDP 不只分片优化器状态；全分片还涉及参数和梯度。

分片存储 → 计算前 all-gather 参数 → 计算后 reduce-scatter 梯度。

19.

TinyML deployments 主要能耗来源是无线通信 Radio communication（例如 BLE/Wi-Fi 上传）

20.

技术栈与硬件无关，可导入模型、自动调度，并能生成用于部署的独立 C/LLVM：TVM

TVM：一套机器学习编译与部署工具链。它可以导入不同框架的模型，自动安排或调优计算，再为目标硬件生成代码；课件提到它能面向 CPU、GPU 和 MCU，并可生成独立的 C/C++/LLVM 部署产物。课件举出的场景包括异构设备和边缘设备部署。简单记：**TVM 帮模型适配、优化并生成适合目标设备的代码。**

XLA：会融合算子、选择高效的数据布局、复用缓冲区。课件建议用于形状稳定的 TensorFlow 模型；动态控制流、频繁改变形状等场景要谨慎。

21.

数据并行 **Data Parallelism** 每台设备保存完整模型，处理不同的数据分片

同步数据并行 **Synchronous Data Parallelism** 每步同步梯度，再让各设备更新模型；通常通过 all-reduce

异步数据并行 **Asynchronous Data Parallelism** 设备无需每步一起等待，常与参数服务器模式相关；可能出现过时梯度

弹性数据并行 **Elastic Data Parallelism** 训练期间参与的 worker 数量可变化，需要重新分片数据并调整学习率

模型并行 **Model Parallelism** 把模型分配到多个设备上计算

层并行 **Layer-wise Parallelism** 把不同层放在不同设备上；前向、反向计算依次流过设备

流水线并行 **Pipeline Parallelism** 把模型分成多个 stage，再用 microbatch 让多个 stage 重叠工作

张量并行 **Tensor Parallelism** 把同一层中的矩阵或维度拆到多台 GPU 上计算

混合并行 **Hybrid Parallelism** 组合数据并行、张量并行、流水线并行等策略

全分片数据并行 **Fully Sharded Data Parallel (FSDP)**分片保存参数、梯度和优化器状态，降低单卡内存占用

联邦学习 **Federated Learning** 客户端本地训练并发送模型更新；课件将其作为隐私保护的分布式训练方式介绍

22.

JAX：jit 是 **just-in-time compilation（即时编译）**：运行时编译函数，以便后续优化执行，同时保留函数原本的计算含义

23.

混合精度训练 mixed-precision training 优先选 **BF16**（动态范围较宽，数值更稳定）；若用 **FP16**，应配合动态 loss scaling，降低溢出或下溢风险。

24.

Amdahl's Law upper bound on speedup 加速比上界 is $S \leq \frac{1}{s + (1-s)/N}$

其中 s 是串行部分比例，N 是处理单元数量；即使 N 无限增大，最大加速比也不超过$\frac{1}{s}$

25.

Hierarchical all-reduce（分层 All-reduce）先在节点内部执行大量归约 reduction，再进行节点间通信，来减少跨节点流量

**Ring all-reduce** 设备按环形与邻居通信；适合大消息，带宽效率好

**Tree all-reduce** 小消息时延较低是另一种集体通信算法

**Parameter server（参数服务器）**灵活，可支持异步训练；服务器可能成为瓶颈

worker 和服务器之间仍要通信

26.

FedAvg 按每个客户端的**本地样本数**加权聚合更新；数据量大的客户端权重更大，这样不会把数据量差异很大的客户端一律当成同等大小。

27.

**Ring** 对大消息的带宽效率较好；**Tree** 对小消息通常延迟更低

28.

JAX 中 pmap 以 SPMD 方式将函数映射到多个设备上

29.

MCU optimizations for depthwise convolution 深度可分离卷积：

普通卷积会让输入通道与输出通道进行组合计算。**Depthwise convolution（逐通道卷积）**则是每个输入通道各自用一个滤波器做空间卷积，不在这一步混合通道；之后通常再接一个 **1×1 pointwise convolution（逐点卷积）**来混合通道。

**预先计算索引或偏移量**：循环中如果反复计算输入、输出位置对应的地址，会浪费指令。提前算好索引，可以减少重复地址计算。

**使用安全且对齐的宽加载**：例如一次加载 8 字节，可能一次取得多个相邻数据，减少逐个读取的开销。前提是内存对齐，且不会越界。

**循环展开**：把循环中的多次迭代展开，减少循环控制开销，并让处理器有更多机会并行执行指令。

**每一行都频繁 malloc/free**：动态申请、释放内存会带来额外开销，也可能造成内存碎片。课件对受限设备更推荐预先分配内存，避免运行时反复分配。

30.

Federated learning client-scheduling practices that reduce round time and bias:

超额邀请 Over-invite，截止前收到前 N 个更新就开始聚合，因为客户端可能掉线或迟到，邀请多于目标数量，再对先返回的足量更新进行部分聚合，可以减少等待 straggler（慢客户端）的时间。课件提到可以 over-select，并设置 deadline、进行 partial aggregation。

按设备或数据类别分层抽样 Stratified sampling，不同设备、地区或数据特征的用户都应有机会参与。分层抽样可以减少样本代表性偏差；课件举例按地理位置、设备类别或数据属性分层。

限制单个客户端的参与频率，防止网络快、性能强的设备被反复选中，而其他客户端长期缺席，有助于参与公平性

31.

在每轮联邦训练中，客户端本地算出模型更新后，可以先**裁剪更新**来限制敏感度，再添加经过校准的**高斯噪声**，然后结合安全聚合上传。由于多轮训练会不断消耗隐私预算，还要记录ε, δ，接近预算上限时停止或调整训练。

在设备端裁剪并加噪，再与安全聚合结合

客户端在本地对更新做范数裁剪并添加校准的高斯噪声，再通过安全聚合提交更新；服务器只看到聚合结果。

多轮训练会不断消耗隐私预算，还要记录ε, δ，接近预算上限时停止或调整训练

传输加密只能保护传输过程；要获得差分隐私保证，还需要裁剪更新、添加校准噪声，并记录ε, δ，接近预算上限时停止或调整训练

32.

TinyML 部署的能效策略包括:

Duty cycling 占空比控制(short active bursts, long sleeps)

Cascaded detection 级联检测 (cheap stage gates a heavier classifier)

Compute more locally to transmit fewer bytes

尽量在设备空闲、充电、连 Wi-Fi 且电量充足时安排训练。

33.

TFLM 在 MCU 上的内存管理最佳实践包括

使用固定 tensor arena，避免在关键推理流程中使用 heap 动态分配

arena 太小，分配张量时会报错；应增大 arena

预分配 Preallocate 并复用缓冲区，可避免反复分配的开销和内存碎片 fragmentation

34.

Pipeline parallelism statements:

有 m 个 stage、b 个 microbatch 时，microbatching 可把 bubble 从约 m-1 降到约 (m-1)/b

1F1B 会交错执行 forward 和 backward，减少 bubble，也能降低激活值的内存压力。

GPipe 是先执行所有 microbatch 的 forward，再执行 backward

增加 microbatch 数通常能降低 bubble、提高利用率，但可能增加延迟或激活值内存压力，因此需要权衡

35.

分层 All-reduce 如何沿着集群的网络层次完成梯度归约和分发

节点内 Reduce-Scatter：先在每台机器内部，把 GPU 的数据归约并分片

节点间 Reduce-Scatter：再通过节点间网络继续归约、分发分片

节点间 All-Gather：节点间交换已归约的分片

节点内 All-Gather：最后在每台机器内部收集分片，使本机各 GPU 得到完整结果

36.

ZeRO 优化器（普通数据并行会在各设备上复制很多训练状态；ZeRO 把这些状态分散存储，减少每张 GPU 的内存占用）各阶段包括:

**Stage 1** 分片存储优化器状态（optimizer states）

**Stage 2** 在 Stage 1 的基础上，也分片梯度（gradients）

**Stage 3** 再把模型参数（parameters）也分片；还可以选择把状态卸载到 CPU 或 NVMe

ZeRO 不会消除所有集体通信。分片后仍需要 all-gather、reduce-scatter 等通信来准备计算和同步梯度；课件指出，分片省内存的代价是增加 gather/scatter 通信

37.

FSDP（Fully Sharded Data Parallel，全分片数据并行）的行为包括:

参数分片平时分布在各 rank 上；某个模块要计算时，先 **all-gather** 该模块的参数。算完后及时释放临时完整参数；反向阶段通过 **reduce-scatter** 汇总并分片梯度。

FSDP 支持混合精度（mixed precision），可减少参数等数据的内存占用。

FSDP 的核心是参数分片存储，并非所有 rank 始终保存完整参数；完整参数是在计算需要时临时收集。

FSDP 的 wrapping/granularity（包裹模块的粒度）会影响参数收集和梯度通信的频率，因而影响通信与计算的平衡。

38.

Reproducibility at scale 同一配置重跑时，尽量保证数据分片、随机过程和软件硬件环境一致，并保存足够状态以便恢复:

为全局和每个 rank 设置随机种子，包括 Python、NumPy、框架和 CUDA；同时使用确定性的数据采样与分片。

建议启用确定性后端，避免或替换已知的非确定性 kernel；若因性能等原因保留，要记录例外。

Checkpoint 除了模型和优化器状态，也应保存随机数状态，以及 dataloader 的 epoch/offset，才能更准确地恢复训练进度。

固定并记录 CUDA、cuDNN、NCCL、驱动和框架版本，也记录代码配置、硬件拓扑和

world size。

39.

减小串行 serial 比例 s / 隐藏通信以提升扩展性的做法包括：

用 gradient bucketing 等方法，让梯度通信和反向计算重叠进行；融合 kernel 也能减少执行开销。这就是“隐藏通信”：计算还在继续时，同时传已经准备好的梯度。

降低梯度精度或压缩梯度，可以减少通信数据量；课件提到压缩时可配合 error feedback，补偿压缩造成的信息损失。

适当增加本地 batch，或谨慎采用延迟/部分同步、异步方案，可以减少同步频率，但要留意收敛和准确率。

40.

内核中的内存带宽优化包括：

循环顺序要配合张量布局，让内层循环尽量读取连续内存。例如课件展示了按 H → W → C 顺序访问 NHWC 数据的连续访问方式。

大量很小的 kernel launch 会增加启动开销，并不天然带来更好的交错执行。

条件允许时原地操作（in-place operation），直接覆盖已有数据，减少额外缓冲区和读写流量；具体能省多少取决于操作。

使用内存池或 arena 预分配、复用缓冲区，减少运行时反复 malloc/free 的开销，并降低内存碎片风险。

Eg2

1.

神经网络训练：参数很多，loss 通常是一个标量，反向模式适合“**很多输入变量、少量输出**”的情况

2.

对于对偶数，单变量函数满足 $f(a+\varepsilon b) = f(a) + f'(a)\varepsilon b$

3.

TFLM 推理时使用预先提供的固定大小 **tensor arena（张量内存区）**，避免在关键路径中动态分配内存。这样内存占用更可预测，也能避免堆碎片；若 arena 不够大，分配张量时会报错，需要增大 arena。

4.

XLA-backed JIT(e.g., JAX/TF) 的主要好处是编译时可以做图级优化：**融合多个算子、选择合适的数据布局、复用内存缓冲区**，从而减少 kernel 启动、数据搬运和内存占用。

5.

配合 -ffunction-sections/-fdata-sections 使用时，哪种链接器选项会从最终二进制文件中删除未使用的代码/ 数据

--gc-sections：链接时回收未被引用的代码或数据段

6.

hierarchical all-reduce sequence：

节点内（intra） reduce-scatter → 节点间（inter） reduce-scatter → 节点间 all-gather → 节点内 all-gather

7.

secure aggregation for FL：

安全聚合用成对掩码隐藏单个客户端的更新；正常情况下，掩码会在总和中相互抵消。若客户端中途掉线，部分掩码可能无法自动抵消，因此需要**阈值式掩码恢复**来处理掉线客户端，使服务器仍能得到聚合结果，而不直接看到各客户端的单独更新。

8.

In TensorFlow 2.x, the default execution style is Eager (imperative) execution with optional tf.function

默认是 **eager（立即执行、命令式）模式**：写下一个操作，它就会立即运行，比较直观，也方便调试。

9.

TensorFlow 1.x 的 **define-then-run** 是先定义完整的**符号计算图**，再通过 **Session** 执行图。定义图时主要是在描述计算流程，操作不会像 eager 模式那样立即执行。

10.

TFLM 中的 **FlatBuffers** 让模型以定义好的二进制结构保存，并支持直接访问数据，不必先做繁重的解析或复制。零拷贝、经过 schema 校验的访问，且无需繁重解析

11.

在 TinyML 系统中，主要能耗往往来自 Radio uplink

12.

CNN 常见融合模式：Conv2D → BatchNorm → ReLU

13.

对于 NHWC 张量，缓存友好的内层循环遍历顺序

NHWC 的维度顺序是 **[Batch, Height, Width, Channels]**。对同一个像素而言，它的各个通道值相邻存放，因此最内层遍历 c，能连续访问内存，提升缓存局部性。若把 c 放在外层或采用随机步长，就会频繁跳过内存位置，访问效率较低。

14.

同步 all-reduce 每一步都设有同步屏障：各 worker 算完本地梯度后一起做 all-reduce，再使用相同的更新继续训练

“陈旧梯度”是异步或延迟同步更典型的风险

15.

参数服务器（**Parameter Server, PS**）采用异步或 stale-sync 方式时，worker 不必像同步 all-reduce 那样每一步都等所有设备到齐，因此能减少慢设备造成的等待，提高吞吐量。代价是服务器收到的梯度可能基于较旧的模型参数，即**陈旧梯度（stale gradients）**

16.

**Stage 1** 分片存储优化器状态（optimizer states）

**Stage 2** 在 Stage 1 的基础上，也分片梯度（gradients）

**Stage 3** 再把模型参数（parameters）也分片；还可以选择把状态卸载到 CPU 或 NVMe

17.

Reduce-scatter（归约并分片）：它把两件事合在一起：先对不同 rank 上对应的数据做归约（通常求和），再把归约结果切成几块分给各 rank。比如 4 个 rank 一起算梯度，reduce-scatter 后每个 rank 只保留归约后梯度的一部分，适合需要分片保存梯度的训练。

18.

TFLM 使用预先提供的固定大小 tensor arena，不会因为空间不足就悄悄改用 heap 动态扩容。初始化时发现 arena 不够，就应失败并报告错误

19.

**MLIR（Multi-Level Intermediate Representation，多层中间表示）**不是一个单独的编译器，而是用来构建编译器的框架。它用不同的 **dialect（方言）**表示不同抽象层级的操作，并支持逐步转换、优化，例如：

**TensorFlow Graph → TF dialect → Linalg dialect → LLVM IR → Machine Code**

20.

Hierarchical all-reduce（分层 All-reduce）

先在每个节点内部归约，再进行节点间通信，从而减少跨节点交换的数据量。按课件流程，是先节点内 reduce-scatter，再节点间 reduce-scatter、节点间 all-gather，最后节点内 all-gather。节点内可用 NVLink；节点间可用 InfiniBand 或 Ethernet。

21.

联邦学习中讨论的一种个性化策略：

客户端共同训练、聚合共享的 backbone（主干网络），而每个客户端的 private head（私有输出头）留在本地，用来适应自己的数据和任务分布，这样既能利用多个客户端共同学到的特征，也能保留本地个性化部分

22.

For non-IID data，哪种聚合策略可以改善收敛

**Non-IID** 表示不同客户端的数据分布不一样，可能类别比例不同、数据不平衡，甚至会随时间变化。把相似客户端分成 cohort 群组，先在组内聚合，再进行后续组合，可以减少“一刀切”聚合对不同数据分布的影响。

23.

在联邦学习的客户端级差分隐私中，添加噪声前用什么限制单个客户端的影响

在客户端级差分隐私中，裁剪会把每个客户端更新的范数限制在设定上限内，从而限制单个客户端对聚合结果的最大影响，也就是界定敏感度。之后再添加校准的高斯噪声，形成 DP 保护。Per-client norm clipping（逐客户端范数裁剪）

24.

在 DDP 中，梯度分桶的主要用途是什么

DDP 的梯度分桶（gradient bucketing）是把多个参数的梯度放进一个 bucket。反向传播会逐层算出梯度；当某个 bucket 的梯度都准备好后，DDP 就可以立刻对它启动 **all-reduce**，同时继续计算后面层的梯度。

这样通信和计算能重叠，减少训练中等待通信的时间。bucket 太小会产生很多小通信消息；太大则要等更多梯度算完才开始通信，所以需要权衡大小。

25.

DL 框架中常用作后备方案、支持 CPU/GPU 集合通信的通用库是

Gloo

26.

TPU 用脉动阵列加速矩阵乘法。把矩阵切成固定大小的 tile，并在边界补零、对齐到阵列适合的尺寸（如 128），可以提高阵列利用率

27.

小消息通常用 Tree，延迟较低；大消息通常用 Ring，带宽利用较好

28.

如果计算图里有相同的子图被重复计算，CSE Common subexpression elimination（公共子表达式消除）会把它识别出来，让多处共用一次计算结果，从而避免重复计算。

29.

代码体积 Code-size 优化方法

**-Os**：编译器进行偏向减小代码体积的优化

**LTO（链接时优化）**：在链接阶段跨编译单元优化，可能减少最终代码体积

**--gc-sections**：配合 -ffunction-sections 和 -fdata-sections，链接时移除未使用的函数或数据段

30.

计算图的优势：

**自动微分**——沿图应用链式法则计算梯度

**全局调度与算子融合**——系统能观察整个计算图，寻找可重排或融合的操作

**内存规划与缓冲区复用**——根据数据何时产生、何时不再使用来安排内存

31.

高效通信的联邦学习更新可以采用

**量化（Quantization）**：用较少比特表示更新，例如 8/4/2-bit

**疏化（Sparsification）**：只发送部分重要更新；误差反馈会保留未发送部分，之后再补偿

**LoRA**：通过低秩适配器只训练、传输较小的增量参数

32.

适合 TPU 脉动 Systolic 阵列的 GEMM 分块包括

TPU 的脉动阵列通过规律的数据流执行矩阵乘法：把矩阵切成固定大小的 tile；边界补零，让 tile 尺寸一致；然后让 **A 矩阵数据向右流动，B 矩阵数据向下流动**，各计算单元边接收数据边做乘加。随机改变 tile 大小会破坏这种规则的数据流

33.

针对非 IID 和噪声的稳健联邦学习聚合可以采用

**截尾均值 / 按坐标取中位数**：属于鲁棒聚合方法，降低异常更新对聚合结果的影响

**更新范数裁剪**：限制单个客户端更新的大小，避免过大的更新主导聚合

**异常值过滤**：识别并过滤离群或不一致的客户端更新

34.

大规模可复现性建议包括

**设置随机种子**，包括 Python、NumPy、框架和 CUDA 的 RNG

**检查点保存随机状态和数据加载进度**，例如 RNG 状态、dataloader 的 epoch/offset，这样恢复训练时能接着原来的状态继续

**固定环境版本**，记录 CUDA、cuDNN、NCCL、驱动和框架版本

非确定性 kernel 可能让结果难以复现；课件建议尽可能避免或替换已知的非确定性操作

35.

TFLM 在 MCU 上的良好内存实践包括

arena 不够时应尽早报错，而不是运行中才发现内存不足

热路径中避免反复 malloc/free，减少堆碎片和不可预测的分配失败

预分配并复用缓冲区，减少内存占用和碎片

每次推理都动态分配会增加运行时开销，也让内存使用更难预测

36.

FSDP（Fully Sharded Data Parallel）通常会

计算前对参数分片做 **all-gather**，让当前模块取得完整参数

模块计算完成后释放完整参数，降低显存占用

反向传播后对梯度做 **reduce-scatter**，完成规约并把梯度分片分回各 rank

37.

流水线并行的调节参数/权衡包括

增加 microbatch 数可以缩短流水线空泡

课件明确说 1F1B 会交错执行前向与反向，降低激活值的显存占用

激活检查点用额外计算换取更低的内存占用

GPipe 是先执行所有前向，再执行所有反向；交错执行的是 1F1B

38.

降低串行比例 serial fraction s 或隐藏通信的策略包括

梯度分桶可以让通信和后续反向计算重叠；融合 kernel 可减少启动和执行开销

低精度或压缩梯度减少每轮传输的数据量；误差反馈会保存被压缩或省略的误差，后续再补偿

增加本地 batch 或减少同步频率能降低通信开销，但可能影响收敛效果

移除健康检查会让故障和性能拖慢更难发现、处理

39.

考虑内存带宽的内核实践包括

循环顺序要配合 NHWC/NCHW 的内存布局，让最内层循环访问连续的数据，减少缓存未命中

安全时用原地操作覆盖输入，能避免额外复制、减少内存流量。前提是后续步骤不再需要原始输入

大量微小 kernel 启动会增加启动开销，并不会自动改善交错执行

内存池或 arena 通过预先分配、重复使用缓冲区，减少运行时分配开销和内存碎片

40.

world size 变化时的弹性分布式训练策略包括

重新计算有效全局 batch size，并相应调整学习率

worker 变化后要重建 process groups，并重新分片数据和 checkpoint 状态

如果精度设置没变，loss scaling 也要保持一致，避免数值行为意外变化

checkpoint 对恢复训练和重新分片很重要；弹性训练需要保存模型、优化器、随机数状态及分片信息，而不是禁用 checkpoint。